

Kubernetes Liveness Probe, Readiness Probe, StatefulSet & DaemonSet — Complete Real-Time Interview Explanation

1. Liveness Probe

What is Liveness Probe?

Liveness Probe checks:

Whether application/container is alive or stuck.

If liveness probe fails:

- Kubernetes restarts container automatically.
-

Why Important?

Sometimes application:

- Stops responding
- Gets deadlocked
- Hangs internally

Container may still be running,
but application becomes unusable.

Liveness probe detects this.

Real-Time Scenario

Java application entered deadlock state:

- Container running
- But API not responding

Liveness probe failed,
so Kubernetes restarted pod automatically.

Example

livenessProbe:

httpGet:
 path: /health
 port: 8080

initialDelaySeconds: 10
periodSeconds: 5

What Each Field Means?

Field	Purpose
path	Health check endpoint
port	Application port
initialDelaySeconds	Wait before first check
periodSeconds	Probe interval

Why **initialDelaySeconds** Important?

Some applications:

- Need time to start

Without delay:

- Kubernetes may restart app too early
-

Why **periodSeconds** Important?

Controls:

- How frequently probe runs
-

Real-Time Troubleshooting

If pod restarting repeatedly:

`kubectl describe pod <pod-name>`

Why?

Events section shows:

Liveness probe failed

Important Interview Point

“Liveness probe ensures unhealthy or stuck applications are automatically restarted.”

How to Explain in Interview

“We use liveness probes to detect application hangs or deadlocks and allow Kubernetes to recover containers automatically.”

2. Readiness Probe

What is Readiness Probe?

Readiness Probe checks:

Whether application is ready to receive traffic.

Important Difference

Liveness Probe

Checks:

Is app alive?

Readiness Probe

Checks:

Is app ready for traffic?

What Happens If Readiness Fails?

Kubernetes:

- Removes pod from service endpoints
- Stops sending traffic

BUT:

- Container not restarted

Real-Time Scenario

Spring Boot app takes:

30 seconds

to fully initialize.

Without readiness probe:

- Traffic routed immediately
- Users get failures

With readiness probe:

- Traffic routed only after app ready.
-

Example

readinessProbe:

httpGet:
path: /ready
port: 8080

Real-Time Benefit

Used heavily during:

- Rolling deployments
 - Zero downtime deployments
-

Important Interview Point

“Readiness probes prevent traffic from reaching partially initialized applications.”

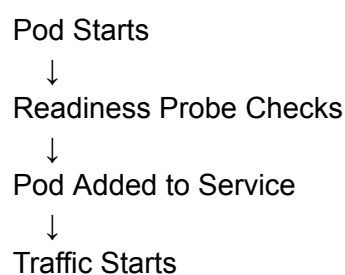
How to Explain in Interview

“We use readiness probes to ensure traffic reaches only healthy and fully initialized application pods.”

Difference Between Liveness & Readiness

Liveness Probe	Readiness Probe
Checks app health	Checks app readiness
Restarts pod on failure	Removes pod from traffic
Detects deadlocks	Prevents failed requests
Self-healing mechanism	Traffic management mechanism

Real-Time Deployment Flow



3. StatefulSet

What is StatefulSet?

StatefulSet is used for:

Stateful applications

Examples:

- Databases
 - Kafka
 - Elasticsearch
 - Redis clusters
-

Why StatefulSet Needed?

Because some applications require:

- Stable identity
- Stable storage
- Ordered startup/shutdown

Deployment cannot provide these guarantees.

Real-Time Problems with Normal Deployment

Pods may:

- Get random names
- Lose storage
- Start in random order

Not suitable for databases.

StatefulSet Features

Stable Pod Names

Example:

mysql-0
mysql-1
mysql-2

Stable Persistent Storage

Each pod gets:

- Dedicated PVC
-

Ordered Deployment

Pods start sequentially:

mysql-0 → mysql-1 → mysql-2

Real-Time Scenario

MySQL replication cluster deployed using StatefulSet because:

- Each DB node required persistent storage
 - Stable hostname needed for replication
-

Example

kind: StatefulSet

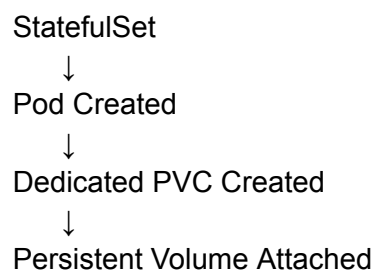
Important Interview Point

“StatefulSet is designed for applications requiring stable network identity and persistent storage.”

How to Explain in Interview

“We use StatefulSets for stateful workloads like databases because they provide stable pod identity, ordered deployment, and persistent storage.”

StatefulSet + PVC Flow



4. DaemonSet

What is DaemonSet?

DaemonSet ensures:

One pod runs on every node.

Why Important?

Some workloads must run on:

- Every worker node
-

Real-Time Use Cases

Log Collection

Examples:

- Fluentd
 - Fluent Bit
-

Monitoring Agents

Examples:

- Node Exporter
 - Datadog Agent
-

Security Agents

Examples:

- Falco
 - Antivirus agents
-

Real-Time Scenario

Prometheus Node Exporter deployed using DaemonSet because:

- Metrics needed from every node
-

What Happens When New Node Added?

DemonSet automatically:

- Deploys pod on new node

Example

kind: DaemonSet

Real-Time Flow

Worker Node 1 → Monitoring Pod

Worker Node 2 → Monitoring Pod

Worker Node 3 → Monitoring Pod

Important Interview Point

“DaemonSets are ideal for node-level services like logging, monitoring, and security agents.”

How to Explain in Interview

“We use DaemonSets for workloads that must run on every Kubernetes node, such as monitoring and logging agents.”

Real-Time Comparison

Deployment	StatefulSet	DaemonSet
Stateless apps	Stateful apps	Node-level services
Random pod names	Stable pod names	One pod per node
Shared behavior	Ordered deployment	Automatic node coverage

Real-Time Kubernetes Production Example

Frontend App → Deployment

MySQL Database → StatefulSet

Prometheus Node Exporter → DaemonSet

Traffic Validation → Readiness Probe

Health Monitoring → Liveness Probe

Most Important Interview Closing Line

“In Kubernetes, probes improve application reliability, StatefulSets handle persistent workloads, and DaemonSets manage node-level operational services like monitoring and logging.”